

CSCE 606 SOFTWARE ENGINEERING · MODULE 08 · LECTURE NOTES

Version Control and Collaboration

The tool everyone uses and few understand.

MODULE OUTCOMES

Explain Git's object model and why it explains the commands.

Use rebase, bisect, and reflog with confidence.

Write commit messages that are useful years later.

Compare branching strategies against team size and release cadence.

Recover from the common disasters.

1 The object model

Most confusion about Git comes from learning commands without the model underneath. The model is small — four object types — and understanding it makes the commands follow.

Object	Contains	Identified by
Blob	File contents. No name, no permissions.	SHA-1 of its contents.
Tree	A directory listing: names mapped to blobs and other trees, with modes.	SHA-1 of the listing.
Commit	A root tree, parent commit(s), author, committer, timestamp, message.	SHA-1 of all of that.
Tag	A named, annotated pointer to a commit.	SHA-1.

Everything is **content-addressed**: the name of an object is the hash of its content. Two identical files anywhere in history are stored once. Two commits with identical trees share the tree.

A COMMIT IS A SNAPSHOT, NOT A DIFF

This surprises people coming from older systems. A commit references a complete tree of the project at that moment, not a set of changes.

Diffs are *computed* on demand by comparing two trees. This is why Git can show a diff between any two commits instantly without walking the history between them, and why merging compares trees rather than replaying patches.

A branch is a file containing a 40-character hash. That is all. Creating a branch writes 41 bytes; deleting one removes a name and nothing else. This is why Git branching is cheap when branching in older systems was an event.

COMMITTS ARE IMMUTABLE; BRANCHES MOVE

A commit's hash is computed over its content *including its parent hash*. Change anything — the message, a file, the parent — and the result is a different commit with a different hash.

So rebase, commit --amend, and cherry-pick never modify a commit. They create new commits and move a branch pointer to them.

The original commits still exist, now unreferenced, until garbage collection removes them — by default after about thirty days.

This is precisely why git reflog can rescue you from almost anything: the commits were never deleted, and the reflog records every position each branch has held.

2 The commands worth learning properly

Command	What it does	When
git bisect	Binary search through history for the commit that introduced a behaviour.	Any time you know it worked before and does not now.
git reflog	Shows every position every branch and HEAD has occupied locally.	Recovering from any mistake.
git rebase -i	Rewrite a sequence of your own commits: reorder, squash, edit, drop.	Cleaning up a messy branch before review. Never on shared history.
git blame	Last commit to touch each line.	Finding the <i>context</i> for a line — the name is unfortunate, the purpose is archaeology.
git log -S'text'	Commits where the number of occurrences of a string changed.	'When was this introduced?' and 'where did this constant come from?'

```
git bisect start
git bisect bad           # HEAD is broken
git bisect good v1.2.0   # this release was fine
# Git checks out the midpoint; test it and report:
git bisect good          # or: git bisect bad
# ... repeat. 1000 commits -> ~10 steps.

# Or fully automated:
git bisect run ./reproduce.sh # exit 0 = good, non-zero = bad
```

bisect run is the highest-value underused tool in everyday practice. The script does not need to be a real test — anything that reproduces the problem and exits with a status will do. Set it going and it will identify the commit while you do something else.

3 Commit messages

Reject orders with a delivery date in the past

The order form accepted any date, so backdated orders reached the warehouse queue and were silently dropped by the scheduler, which filters on date $\geq$ today. Users saw "order placed" and nothing arrived.

Validation is at the API boundary rather than in the form, because the mobile client posts directly to the API.

Fixes #1423.

Part	Rule
Subject	What changed, in the imperative mood, under about 50 characters. 'Reject orders...' not 'Rejected' or 'Rejects'. It completes the sentence 'If applied, this commit will...'
Blank line	Required. Tooling treats the first line specially.
Body	Why. The diff already shows what. Explain the problem, the constraint, the rejected alternative.
Footer	Issue references, co-authors, breaking-change notes.

WRITE FOR THE ARCHAEOLOGIST

In three years, someone will run `git blame` on a line of code, land on your commit, and need to decide whether the line can safely be changed.

They can read the diff perfectly well. What they cannot reconstruct is **why** — which bug prompted it, which constraint forced the odd approach, which obvious alternative was tried and failed.

If your message does not record that, the knowledge is gone. There is no other durable record: the ticket may be in a system that has been replaced, the discussion was in a chat that has been purged, and the people have left.

This is the only documentation that cannot rot, because it is attached permanently and immutably to the change it describes. 'Address PR feedback' as a message throws that away.

4 Branching strategies

Strategy	Mechanics	Suits
Trunk-based	Everyone commits to main, multiple times daily. Incomplete work is hidden behind feature flags. Releases are tags or short-lived release branches.	Continuous delivery. The practice DORA associates with the highest-performing teams.

Strategy	Mechanics	Suits
GitHub flow	Branch from main, open a pull request, review, merge, deploy. Branches live hours to days.	Most teams, most of the time. Simple and adequate.
Git flow	Permanent develop and main, plus feature, release, and hotfix branch types with prescribed merge directions.	Software with versioned releases supported in parallel — installed products, libraries with long-term support branches. Substantial ceremony , and its author has since written that it is poorly suited to continuously delivered web applications.

THE PROBLEM IS BRANCH LIFETIME, NOT THE MERGE TOOL

A branch that lives three weeks diverges from main for three weeks. The resulting merge is large, unfamiliar, and conflict-ridden — and the difficulty grows worse than linearly with the branch's age, because conflicts interact.

DORA's consistent finding is that high-performing teams integrate to trunk at least daily, with branches typically living less than a day. This correlates with both faster delivery and greater stability — which is not the trade-off people expect.

The obvious objection is work that takes a week to complete. **The answer is feature flags**: merge the incomplete work in a disabled state, integrate continuously, and enable it when it is ready.

So 'how do I avoid merge hell' and 'how do I deploy safely' turn out to have the same answer. Module 10 develops it.

5 Recovery

Situation	Recovery
Committed to the wrong branch.	git reset HEAD~ (keeps the changes), switch branch, commit again.
Deleted a branch you needed.	git reflog, find the last commit hash, git branch <name> <hash>.
A rebase went wrong.	git reflog, find the pre-rebase position, git reset --hard <hash>.
Committed a large file by accident.	If unpushed, amend or rebase it out. If pushed, git filter-repo and a coordinated force-push.
Committed a secret.	Rotate the secret. Immediately. Rewriting history does not help: it was pushed, it may be cloned, forked, cached by the host, or indexed. Treat it as public, because it is.

Almost everything is recoverable for about thirty days through the reflog. The exception is work that was never committed, which Git has no record of. **Commit early and often**; a messy history can be cleaned up with interactive rebase before review, and lost work cannot be cleaned up at all.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Pro Git (free book)	Chapter 10 ('Git Internals') is the one that makes everything else make sense. Read it before the command chapters.
Git from the Bottom Up (free)	Builds the model from the object store upward. Short, and the best explanation of why rebase works the way it does.
Conventional Commits	A structured message format that enables automated changelogs and version bumping. Useful when adopted consistently.
DORA — trunk-based development	The evidence behind §4's claim about branch lifetime.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Explore the object store directly: commit a file, then use `git cat-file -p` to walk from the commit to its tree to the blob. Confirm the blob's hash is the hash of the contents.
- 2 Create a branch, make commits, delete the branch, then recover it using `git reflog`.
- 3 Introduce a bug ten commits back in a test repository, then find it with `git bisect`. Then automate it with `bisect run`.
- 4 Take a branch with five messy commits and clean it with `git rebase -i`: squash, reorder, and rewrite the messages.
- 5 Find a commit in a real open-source project whose message explains *why*. Find one that does not. Assess how much harder the second makes a hypothetical change.
- 6 Measure branch lifetime in a project you can see: for twenty merged pull requests, record the time between first commit and merge, and plot it against the number of conflicts.
- 7 Deliberately create a merge conflict, resolve it, and then examine what Git recorded about the resolution.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name Git's four object types and say what identifies each.

- 2 Why is a commit a snapshot rather than a diff, and what two things does that explain?
- 3 Why does rebase not modify commits, and why does reflog work?
- 4 What does git bisect run do, and why is it underused?
- 5 What must a commit message contain that the diff cannot provide?
- 6 Why are long-lived branches the problem, and what makes short ones possible for week-long work?
- 7 What is the one thing Git cannot recover, and what should you do about a committed secret?